

Access Checking on Specializations

ISO/IEC JTC1 SC22 WG21

Document Number: P0692R1

Matt Calabrese (metaprogrammingtheworld@gmail.com)

2017-11-10

Abstract

This paper attempts to address a long-standing hole in the ability for developers to specialize templates on their `private` and `protected` nested class-types. It also identifies an implementation divergence between compilers.

Abstract Code Example

To be clear about what is being discussed, the following code is a minimal example:

```
template<class T>
struct trait;

class class_ {
    class impl;
};

// Not allowed in standard C++ (impl is private)
template<>
struct trait<class_::impl>;
```

It is important to note that even though the above specialization of `trait` is not allowed according to the standard, it **builds with all compilers that were tested**, including various versions of gcc, clang, icc, and msvc. Already, for the sake of standardizing existing practice, one might argue that this should be allowed. Though this specific code seems to have consistent acceptance between implementations, similar code that deals with *partial* (as opposed to *explicit*) specializations is permitted in some compilers while disallowed in others. This paper presents rationale for why code such as this is desirable along with options for how to adjust the language pending the committee's agreement that specializations such as this should be sanctioned.

Revision History

Revision 1

- Added wording after acceptance of "Option A" by EWG in Albuquerque, November 2017.
- Added Jens Maurer to the Acknowledgments section for consultation during wording specification.

Motivating Case

From the minimal code above, it may not be clear why anyone may desire this functionality, but it does come up in practice and is how the differences in compiler behavior were discovered. For example, in development of a standard library proposal^{[1][2]}, I frequently encountered the desire to specialize namespace-scope traits on a `private` nested type of a `struct`. This is due to function objects of the library frequently returning an instance of an *unspecified* type that needs to model a specific concept with associated traits. If this sounds rather abstract, consider the more familiar case of a `Range` type that may have a `private` iterator type. Because this nested type is `private`, it is not possible to specialize `iterator_traits` on it directly.

Implementation Divergence

While all tested compilers allow the non-standard specialization seen in this paper's abstract, something similar but with a partial specialization uncovers implementation divergence. Consider the following:

```
template<class T>
struct trait;

class class_ {
    template<class U>
    struct impl;
};

// Not allowed in standard C++ (impl is private)
// Not allowed in clang, icc
// Allowed in gcc, msvc
template<class U>
struct trait<class_::impl<U>>;
```

Workarounds

A developer who is not a language lawyer may incorrectly believe that there is an obvious, standard solution to this problem: declare the trait to be a `friend` of `class_`. Of course, this will not actually work because declaring a specialization of a trait to be a `friend` does not mean that the declarator's *template argument list* can refer to `private` members of the class. All that such a `friend` declaration means is that the *definition* of the trait would have access. In existing C++, if the trait, itself, were nested in a hypothetical type `foo`, then `foo` could be befriended and we'd be able to directly specialize the trait. Because the trait is at namespace scope, this is not an option.

An actual workaround is to either make `impl` `public`, or make an alias of it `public`, or put an alias or `impl` itself in a hidden details namespace. All of these options may be considered suboptimal by some developers as they require directly exposing a type that is [arguably] most-naturally a `private` nested type.

Related Issues

James Dennett discovered a related issue^[3] submitted by John Spicer in 1999 that attempts to address part of the problem (though only for *explicit* specializations). Resolution of this issue in 2002 was to consider this lack of ability to declare a specialization to be NAD, however there is still no direct way to accomplish what is desired. As well, as was described earlier, the resolution from the issue was never actually implemented in gcc, clang, icc, or msvc, so there may be reason to reconsider that resolution.

Proposal

Assuming that the committee feels this is a problem worth solving, this paper presents four possible solutions to consider, with a preference for one of the alternatives. A table comparing user code corresponding to each option is presented after their enumeration.

Option A

The first option is to standardize a generalization of existing practice and simply allow an explicit or partial specialization to ignore access when the `template` being referred to is not dependent on a `template` parameter. Very important to note is that this does not grant the *definition* of the template any privileged access that it wouldn't already have had. Because it would not affect access when the `template` itself is *dependent*, it would not change the behavior of existing, well-formed code (specifically, it would not break specializations that rely on access playing a role in SFINAE in a partial specialization, which is a common pattern when creating traits).

"Option A" is somewhat consistent with existing explicit instantiation rules and also with the behavior of explicit specializations as it exists in all of the tested compilers, however this does contradict the resolution of issue 182^[3].

Option B

The second option is to make it such that if a `class` `class_` with a `private` nested type `impl` declares a specialization `trait<class_::impl>` to be a friend, then that specialization can be declared and defined outside of the `class`.

One drawback of "Option B" is that it gives the *definition* of the trait privileged access to `class_` when it otherwise may not be necessary, though subjectively, this is a minor drawback.

Option C

The third option is very similar to the second, except the `friend` declaration appears in the nested class itself as opposed to the type(s) that it is nested in. This has some similar drawbacks and is also strange in that the specialization does not need privileged access of the *nested* `class` for the declaration to be valid, but rather, it needs privileged access to the type that *contains* it. This option is included because an informal poll of a very small set of C++ programmers did favor it.

Option D

The final option is to allow a `template` (as opposed to a *specialization* of a `template`) to be specified as a friend. This would be a new kind of `friend` declaration and would be provided by specifying *only* the `template` name in the `friend` declaration rather than a specialization of that `template`. This would not give the *definition* of the trait any privileged access at all, which may be considered a positive aspect of this choice. Of course, privileged access can still be granted via a normal `friend` declaration if it is actually desired.

Comparison of Options

Below is a side-by-side comparison of all of the options described above.

Option A	Option B	Option C	C
<pre>template<class T> struct trait; class class_ { template<class U> struct impl; }; // This would just work. // Tested compilers already allow // explicit specializations. // gcc and msvc already allow // partial specializations. template<class U> struct trait<class_::impl<U>>;</pre>	<pre>template<class T> struct trait; class class_ { template<class U> struct impl; template<class U> friend struct trait<impl<U>>; }; // Works because of the friend declaration. // Side-effect: definition now also has // privileged access to class_. template<class U> struct trait<class_::impl<U>>;</pre>	<pre>template<class T> struct trait; class class_ { template<class U> struct impl { friend struct trait<impl>; }; }; // Works because of the friend declaration. // Side-effect: definition now also has // privileged access to class_::impl. template<class U> struct trait<class_::impl<U>>;</pre>	<pre>template<class T> struct trait; class class_ { template<class U> struct impl; friend trait; }; // Works because of // Definition does r template<class U> struct trait<class_::</pre>

Preferred Solution

The author of this paper is most in favor of "Option A". Behavior like this has existed in commonly-used compilers without problem for at least 15 years and it doesn't require adding a new kind of feature to the language that only experts would know about. It is also somewhat of a safer choice compared to the alternatives as it does not require altering specialization behavior or access in novel ways that have yet to be explored.

Wording

Wording for "Option A":

Note: This wording is relative to N4700 (Pre-Albuquerque, 2017-10-16).

Add a paragraph to §17.6.5 Class template partial specializations [temp.class.spec] after paragraph 9:

The usual access checking rules do not apply to non-dependent names used to specify template arguments of the simple-template-id of the partial specialization. [Note: The template arguments may be private types or objects that would normally not be accessible. Dependent names cannot be checked when declaring the partial specialization, but will be checked when substituting into the partial specialization. — end note]

Add a paragraph to §17.8 Template instantiation and specialization [temp.spec] after paragraph 5:

The usual access checking rules do not apply to names in a declaration of an explicit instantiation or explicit specialization, with the exception of names appearing in a function body, default argument, base-clause, member-specification, enumerator-list, or static data member or variable template initializer.

~~[Note: In particular, the template arguments and names used in the function declarator (including parameter types, return types and exception specifications) may be private types or objects that would normally not be accessible. — end note]~~

Remove §17.8.2 Explicit instantiation [temp.explicit] paragraph 14:

~~The usual access checking rules do not apply to names used to specify explicit instantiations. [Note: In particular, the template arguments and names used in the function declarator (including parameter types, return types and exception specifications) may be private types or objects which would normally not be accessible and the template may be a member template or member function which would not normally be accessible. — end note]~~

Acknowledgments

Thanks to James Dennett and Richard Smith for their interest and research. Also thanks to Jens Maurer for help with wording.

References

[1] Matt Calabrese: "A Single Generalization of std::invoke, std::apply, and std::visit" P0376R0 <http://www.open-std.org/JTC1/SC22/WG21/docs/papers/2016/p0376r0.html>

[2] Matt Calabrese: "Call: A Library that Will Change the Way You Think about Function Invocations" <https://www.youtube.com/watch?v=Fjw7NjndQ50&list=PLTXJhw4sOAviW1OdRgPlxU5m5GU2UzJiJ>

[3] C++ Standard Core Language Issue 182. "Access checking on explicit specializations" http://www.open-std.org/jtc1/sc22/wg21/docs/cwg_closed.html